A Competitive Programming Field Guide

Bit Manipulation

for Competitive Programming

From First Principles to Contest-Ready Technique

About this guide

What you will learn

Every standard bit-manipulation idea used in competitive programming: the fundamental bit operations, the famous one-liners built on `x & (x-1)` and `x & (-x)`, popcount and its many faces, XOR problem patterns, bitmasks as sets, submask enumeration, bitmask dynamic programming (TSP, assignments, SOS DP), binary tries, and `std::bitset` tricks.

Prerequisites

Working knowledge of C++ (variables, loops, functions, vectors), basic Big–O reasoning, and high-school algebra. Everything about binary numbers is built from scratch inside.

Conventions

Code is C++17. Bits are numbered from 0 at the least-significant end. Examples print 8-bit patterns for readability, while real programs use 32- or 64-bit integers.

October 17, 2026

Contents

1 Introduction **3**

2 Prerequisites **3**

2.1 Positional number systems . . . 3
2.2 Bits, bytes, and integer types in C++ . . . 4
2.3 Two's complement: how negatives are stored . . . 5
2.4 How C++ evaluates bitwise expressions . . . 5
2.5 Background assumptions . . . 6

3 The Bitwise Toolbox **6**

3.1 The four logical operators . . . 6
3.2 Shifts: moving bits around . . . 7
3.3 The fundamental operations on single bits . . . 7
3.4 Mask construction and bit fields . . . 8

4 Classic One-Liner Techniques **9**

4.1 Clearing the lowest set bit: `n & (n-1)` . . . 9
4.2 Isolating the lowest set bit: `n & (-n)` . . . 9
4.3 Counting set bits: popcount in all its forms . . . 10
4.4 Positions, not counts: ctz, clz, and integer log2 . . . 11
4.5 Scaling and rounding by powers of two . . . 12
4.6 Swapping and reversing bits . . . 13
4.7 Gray codes: neighboring states, one bit apart . . . 14
4.8 XOR of a range in O(1) . . . 14

5 XOR Problem Patterns **15**

5.1 The single number: everything twice except one . . . 15
5.2 Two unique numbers: partition by a set bit . . . 15
5.3 The missing number . . . 16
5.4 Every element k times except one . . . 16
5.5 Prefix XOR: range queries and subarray counting . . . 17
5.6 Nim and Grundy numbers: XOR as game theory . . . 18
5.7 The binary trie: maximum XOR queries . . . 18
5.8 Per-bit contribution: sum of XOR over all pairs . . . 20

6 Bitmasks as Sets **20**

6.1 Set algebra on integers . . . 20
6.2 Enumerating subsets of a universe . . . 21
6.3 Enumerating submasks of a mask . . . 22
6.4 Inclusion–exclusion over bitmasks . . . 22

7 Bitmask Dynamic Programming **23**

7.1 The pattern . . . 23
7.2 Traveling Salesman Problem . . . 24
7.3 The assignment problem . . . 24
7.4 The lowest-bit trick against double counting . . . 25
7.5 Broken-profile DP: masks across a frontier . . . 26
7.6 SOS DP: sums over submasks in near-linear time . . . 26

8 std::bitset **27**

8.1 The API . . . 27
8.2 Application: reachability and transitive closure . . . 27
8.3 Application: sieve and dp acceleration . . . 28

9 Pitfall Compendium **29**

10 Cheat Sheet and Practice Roadmap **30**

10.1 The one-page cheat sheet . . . 30
10.2 Practice roadmap . . . 31
10.3 Closing advice . . . 32

1 Introduction

Competitive programming constantly rewards contestants who can think in binary. A surprising number of problems that look hopeless with ordinary arithmetic become one-liners once the numbers are viewed as strings of bits. Sets of at most twenty elements fit inside a single integer, where they can be copied, merged, intersected, and passed to functions at the speed of a single machine instruction. Recurrence-heavy counting problems collapse into *bitwise DP* over subsets. Sequences of numbers hide pairings that only XOR can expose. Even graph problems quietly become fast when adjacency rows are stored as machine words and operated on in parallel. This guide collects all of these standard ideas in one place, explains why each one works, and shows how to write them in contest-quality C++.

Bit manipulation is not a standalone topic that lives in isolation; it is a *layer* over the rest of algorithmic problem solving. That is why this document spends its first half building solid foundations: how integers are actually stored, what two's complement really means, how the C++ operators behave on signed and unsigned values, and where the undefined-behavior traps are hiding. Only after that layer is solid do we climb to the famous identity tricks, the XOR problem family, bitmask dynamic programming, and `std::bitset` acceleration. Each technique is presented the same way: motivation first, a proof or intuition second, correct code third, and a note on complexity and common mistakes last.

The techniques in this guide appear constantly on major judges. The CSES problem set has an entire *Introductory Problems* and *Mathematics* section built on these ideas; Codeforces Div. 2 B/C rounds regularly feature XOR-constructive tasks; AtCoder's educational DP and ABC contests use bitmask DP as a core tool. If you plan to work through those problems, the roadmap in Section 10 gives a curated problem list matched to each section of this document.

How to read this guide. Sections 2 through 3 are the foundations: read them in order if binary numbers still feel shaky. Sections 4 and 5 are the heart of the standard toolbox and deserve careful, complete reading. Sections 6–8 cover the "big machinery" (masks as sets, bitmask DP, bitsets); they can be read independently, but each builds on the one-liners from Section 4. Section 9 is a consolidated gallery of traps worth revisiting before every contest, and Section 10 closes with a cheat sheet and a practice roadmap.

2 Prerequisites

Everything in bit manipulation rests on four pieces of background knowledge: how positional number systems work, how integers are laid out in memory, how two's complement encodes negative numbers, and how the C++ language evaluates bitwise expressions. This section builds each of those pieces from zero. If you already know how to convert between bases and why `-x` equals `~x + 1`, skim it once anyway: the *pitfalls* discussed here reappear later as contest-time bugs, and knowing them early will save you hours of debugging.

2.1 Positional number systems

A positional number system writes a number as a row of digits, where each digit carries a different weight depending on its position. In base b, a string $d_k d_{k-1} \dots d_1 d_0$ represents the value

$$d_k \cdot b^k + d_{k-1} \cdot b^{k-1} + \cdots + d_1 \cdot b + d_0.$$

Decimal is base ten with digits 0–9; binary is base two with digits 0 and 1 (called *bits*); hexadecimal is base sixteen with digits 0–9 and `A`–`F` standing for ten through fifteen. Computers store integers in binary, programmers write them in hex because each hex digit encodes exactly four bits, and humans think in decimal: fluency in all three, and in converting between them, is the first prerequisite of this entire topic.

The conversion skill you need most often is *decimal to binary*. Repeatedly divide by two and collect the remainders: dividing 43 by two gives remainders $1, 1, 0, 1, 0, 1$, so $43 = \texttt{101011}_2$, which you can check as $32 + 8 + 2 + 1$. The reverse direction is even more mechanical: read the binary string and add the powers of two under each 1. Since $16 = 2^4$, hexadecimal is the "compressed" view of binary: group bits in fours from the right, then replace each group with one hex digit. For instance $\texttt{10110011}_2 = \texttt{1011 0011}_2 = \texttt{B3}_{16}$. C++ accepts binary literals such as `0b1011` and hex literals such as `0xB3`, and `std::bitset` or `std::hex` will print values in either base when you debug.

Table 1: Powers of two worth memorizing. Contest constraints are usually powers of two or round decimal numbers, and knowing the nearest power of two by heart speeds up every estimate.

k	2^k	k	2^k
0	1	11	2 048
1	2	12	4 096
2	4	16	65 536
3	8	20	1 048 576
4	16	24	16 777 216
5	32	28	268 435 456
8	256	30	1 073 741 824
9	512	31	2 147 483 648
10	1 024	32	4 294 967 296

Table 1 lists the powers of two that show up over and over. Two rough anchors are worth internalizing: $2^{10} \approx 10^3$, so $2^{30} \approx 10^9$ and $2^{20} \approx 10^6$. When a problem says $n \le 10^9$, you instantly know the input fits in a 32-bit signed integer; when it says "all 2^n subsets" with $n \le 20$, you know a million states are involved and bitmask DP is probably intended. These unit conversions between decimal constraints and binary structure are exactly how setters hide hints in the constraints.

2.2 Bits, bytes, and integer types in C++

Memory is organized into *bytes*, each holding eight bits. The byte is the atom of addressing, but arithmetic happens on wider chunks: on modern platforms C++ gives you at least `short` of 16 bits, `int` of 32 bits, and `long long` of 64 bits, each in a signed and an unsigned flavor. A *bit numbering* convention is needed the moment we talk about positions: bit 0 is the least significant bit (the "rightmost" in the usual written form), bit 31 is the most significant bit of a 32-bit integer, and so on up to bit 63 for 64-bit integers. Bitwise operations act on *all* bits of the operand simultaneously, which is what makes them fast.

Table 2: Integer types you will actually use in contests, with the sizes and ranges guaranteed by C++17 on typical 64-bit judges.

Type	Bits	Signed range	Typical use in CP
`int`	32	$-2^{31} \dots 2^{31}-1$	default integer
`unsigned int`	32	$0 \dots 2^{32}-1$	masks, hashing
`long long`	64	$-2^{63} \dots 2^{63}-1$	big sums, products
`unsigned long long`	64	$0 \dots 2^{64}-1$	64-bit masks
`int8_t/int16_t`	8/16	fixed width	memory-tight states
`size_t`	64	$0 \dots 2^{64}-1$	indices, container sizes

Table 2 summarizes the types. Two habits will prevent most integer bugs before they happen. First, prefer signed `long long` for any arithmetic that can exceed 2×10^9, because `int` overflow

is silent and wraps around. Second, when writing masks and shifts, spell the constant width you mean: `1 << i` shifts a 32-bit `int` even if the destination is 64-bit, so for 64-bit work you must write `1LL << i`. The fixed-width aliases `int32_t` and `int64_t` from `<cstdint>` make the width explicit and cost nothing.

2.3 Two's complement: how negatives are stored

Unsigned integers are trivially the plain base-two value of their bits. Signed integers use the *two's complement* encoding, which assigns bit i the weight $+2^i$ for i below the top bit and weight -2^{w-1} to the top bit of a w-bit integer. In an 8-bit world, `00000101` is $+5$, while `11111011` evaluates as $-128 + 64 + 32 + 16 + 8 + 2 + 1 = -5$. The top bit alone therefore tells you the sign, non-negative numbers look exactly like their unsigned versions, and the encoding forms one continuous circle: counting up through zero runs smoothly into the negative range.

Two's complement earns its name from a beautiful identity: for any x, the encoding of $-x$ is the encoding of $\sim x$ (all bits flipped) plus one, i.e. $-x = \sim x + 1$. Intuitively, $x + (\sim x) =$ `1111...1`, the all-ones pattern which is -1 in two's complement; adding one more to $\sim x$ therefore reaches 0 from x, which is precisely $-x$. Two consequences follow immediately and will be used later as tools: subtracting is adding a negation, and `x & (-x)`—the lowest set bit of x, our workhorse in Section 4—works uniformly for positive x because the borrow structure of $x - 1$ complements x exactly at its lowest set bit.

The encoding is *asymmetric*: a 32-bit signed integer holds $2^{32} - 1$ different values, which cannot be split evenly between negative and non-negative, so the negative side gets one extra value. Hence `INT_MIN = -2147483648` has no positive partner: `INT_MIN` $= -$`INT_MIN`, and negating it is undefined behavior. For the same reason, the absolute value function `abs(INT_MIN)` is broken, and any algorithm that negates user input (like `x & (-x)`) needs care if `INT_MIN` can occur. Casting through `unsigned` or working in 64 bits removes the trap.

Finally, converting between widths (*sign extension*) copies the sign bit into all new high-order bits: the 8-bit pattern of -5 becomes the 32-bit pattern of -5 by replicating ones on the left. Unsigned widening, in contrast, pads with zeros. This is why mixing signed and unsigned in one expression is dangerous: C++ converts the signed operand to unsigned, so `-1 < 1u` is `false`, because -1 becomes the huge value 4294967295. Rule of thumb: keep comparisons within one signedness, and turn on warnings (`-Wall -Wextra`); compilers catch most sign-comparison mistakes.

2.4 How C++ evaluates bitwise expressions

C++ applies bitwise operators to the *value* of the operands after the usual arithmetic conversions, and precedence decides how expressions group. Getting precedence wrong is the single most common source of wrong-bit answers, because bitwise operators sit in an unintuitive place: *below* comparison operators. The classic accident is `x & 1 == 0`, which parses as `x & (1 == 0)` $=$ `x & 0` $= 0$, silently turning every input into "even". Table 3 shows the ordering you must internalize; the safest habit, though, is to parenthesize every bitwise subexpression unless it is a bare `a & b`.

One more mechanical detail is *integer promotion*: any operand narrower than `int` (a `char`, a `short`, a bool) is promoted to `int` before the operation. Promotions are harmless for `&`, `|`, and `^` but bite with `~`: `~x` of an 8-bit `char` first becomes a 32-bit `int` and flips 32 bits, not 8, so the high bits hold garbage from the promotion rather than zeros. Comparing the result of `~x` against small constants or storing it back into the same narrow type usually hides the difference, but masks built with `~` should always be built in the width you intend, e.g. `~0u` for a 32-bit all-ones mask.

Table 3: C++ operator precedence relevant to bit manipulation, from highest to lowest. Note that all bitwise operators bind more loosely than arithmetic and (except shifts) more loosely than comparisons.

Level	Operators	Associativity
1	`! ~ -` (unary) `++ --`	right to left
2	`* / %`	left to right
3	`+ -`	left to right
4	`<< >>`	left to right
5	`< <= > >=`	left to right
6	`== !=`	left to right
7	`&`	left to right
8	`^`	left to right
9	`|`	left to right
10	`&&`	left to right
11	`||`	left to right
12	`?:` then `= += &=` etc.	right to left

2.5 Background assumptions

This guide assumes you can already do four things, all standard from any introductory algorithms material. You can reason with Big–O: we will repeatedly compare an $O(n \cdot 2^n)$ bitmask DP against a naive $O(n!)$ search, and you should understand why the first is viable for $n \leq 22$ while the second is not. You can write recursion and understand recursion trees, because subset generation and trie walks are recursive at heart. You know elementary dynamic programming—states, transitions, base cases, and order of evaluation—because bitmask DP is simply DP where the state is an integer whose bits encode a subset. And you know basic graph representation, since the flagship bitmask DP example is the Traveling Salesman Problem over an adjacency matrix.

None of these need to be expert-level. What matters is that phrases like "define `dp[mask]` as the answer for exactly the set of vertices in `mask`" do not scare you. If they do, read the opening of Section 7 slowly; it re-derives the pattern gently. Conversely, deep background is *not* required from number theory or linear algebra: bit manipulation is self-contained, and the few facts borrowed from those fields are proved inline where they are used.

3 The Bitwise Toolbox

This section introduces the six bitwise operators of C++, the algebraic identities they satisfy, and the small vocabulary of operations (get a bit, set a bit, clear a run of bits, extract a field) from which every later technique is assembled. Read it with pencil and paper: rewriting the examples in binary yourself is the fastest way to make the operations feel native rather than memorized.

3.1 The four logical operators

Definition 3.1 (Bitwise operators)**.** For each bit position independently, the operators combine the two corresponding bits of the operands: `a & b` (AND) produces 1 only where *both* operands have 1; `a | b` (OR) produces 1 where *at least one* operand has 1; `a ^` (XOR) produces 1 where the bits *differ*; and `~a` (NOT, unary) flips every bit.

A worked example makes the "in parallel on every bit" rule concrete. Take $a = 43 = \texttt{00101011}_2$ and $b = 27 = \texttt{00011011}_2$. Then `a & b` keeps only the columns where both have ones: $\texttt{00001011}_2 = 11$. `a | b` merges the columns: $\texttt{00111011}_2 = 59$. `a ^ b` marks the differing columns: $\texttt{00110000}_2 = 48$. And `~a` in 32-bit form is $\texttt{11010100}_2$ of the low byte extended with

Table 4: Truth tables of the four logical operators, applied independently to every bit position.

a	b	a&b	a|b	a^b
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

a	~a
0	1
1	0

all ones above it, namely -44 as a signed value. Practicing this column-by-column reading until it is automatic is the point of the example: every technique later in this guide is just several of these columns interacting.

Each operator obeys algebraic laws that turn puzzle-style problems into routine manipulations. AND and OR are commutative, associative, distributive over each other, and idempotent ($a\&a = a$, $a|a = a$); together with the identities $a\& \sim a = 0$, $a|0 = a$, $a\&$`1111...1` $= a$ they reproduce Boolean algebra, and De Morgan's laws hold bitwise: $\sim(a\&b) = \sim a\,|\, \sim b$ and $\sim(a|b) = \sim a\& \sim b$. XOR is commutative and associative too, but its defining property is $a \oplus a = 0$ and $a \oplus 0 = a$: XOR is *addition of subsets mod two*. Every element XORed an even number of times cancels, which is exactly the mechanism behind the classic "find the unique element" problems of Section 5.

Why XOR is special

XOR is its own inverse: $(a \oplus b) \oplus b = a$. This single identity explains half of the XOR folklore in competitive programming: reversible state XOR, the disappearing-pair trick, prefix XOR queries, Gray codes, and even the correctness of the swap-without-a-temporary hack. When you see XOR in a problem, immediately ask: *which quantities appear an even number of times and cancel?*

3.2 Shifts: moving bits around

The shift operators move every bit of their left operand by k positions. A left shift `x << k` fills the vacated low positions with zeros; as long as no set bit falls off the top, it computes exactly $x \cdot 2^k$. A right shift `x >> k` drops the k lowest bits; on *unsigned* types the vacated high positions are filled with zeros, so it computes $\lfloor x/2^k \rfloor$. On *signed* types the fill copies the sign bit (*arithmetic shift* on all mainstream compilers), so right-shifting a negative value keeps it negative and behaves like division rounding toward $-\infty$: e.g. `-7 >> 1` $= -4$, not -3. The asymmetry matters: shifting is a cheap division only when you know your values are non-negative or you specifically want floor division.

Shifts are the standard way to *build* constants and masks. `1 << i` is a single set bit at position i; `(1 << k) - 1` is a run of k ones (for example `(1 << 4) - 1` $= 1111_2 = 15$); and `~((1 << k) - 1)` is that run inverted, i.e. all ones above position $k-1$. Combining a run shifted into place is the universal recipe for "isolate the bits from l to r" that Section 3.3 formalizes. Two warnings from Section 2 apply to every shift you write: shifting by a count equal to or larger than the width of the left operand is undefined behavior, and the left operand determines the width, so 64-bit results need `1LL`/`1ULL` constants or an explicit cast.

3.3 The fundamental operations on single bits

Nearly everything in this guide reduces to six primitives acting on bit i of an integer n. They are small enough to memorize and important enough to deserve exact wording, so we give them in one block with comments, then walk through the reasoning once. Throughout, "position i" means bit i, counting from zero at the least significant end, and the code assumes $0 \le i \le 31$ for `int`.

```cpp
// n = 0b101100 (44), i = position of the bit, 0 = least significant
bool getBit(int n, int i)    { return (n >> i) & 1;        }
int  setBit(int n, int i)    { return n | (1 << i);        }
int  clearBit(int n, int i)  { return n & ~(1 << i);       }
int  toggleBit(int n, int i) { return n ^ (1 << i);        }

int updateBit(int n, int i, bool v) {       // write value v into bit i
    return (n & ~(1 << i)) | (v << i);      // clear first, then OR in v
}
```

Listing 1: The six fundamental bit operations.

The reasoning is identical for all six. To *read* bit i, shift n right by i so that the interesting bit lands at position 0, then AND with 1 to erase everything else. To *modify* bit i, build the one-hot mask `1 << i`. OR-ing a one-hot mask forces that bit to 1 and cannot touch others (OR with zero is identity); AND-ing with the inverted mask `~(1 << i)` forces it to 0 (AND with one elsewhere is identity); XOR-ing the mask flips exactly that bit because $b \oplus 0 = b$ and $b \oplus 1 = \sim b$ at every other position the mask has zeros. To *write* an arbitrary value v, you cannot just OR it in unless it is zero, so the update operation clears the slot first and then ORs v shifted into place.

Example 3.1. Let $n = 44 = \texttt{00101100}_2$ and $i = 3$. The mask is `00001000`. `getBit(44,3)` computes $\texttt{1011}_2 \& 1 = 1$, so the bit is set. `setBit` gives $\texttt{00101100} \,|\, \texttt{00001000} = \texttt{00101100}$ (unchanged, it was already set). `clearBit` gives $\texttt{00101100}\&\texttt{11110111} = \texttt{00100100}_2 = 36$. `toggleBit` also gives 36 here. Now $i = 1$: `setBit(44,1)` $= \texttt{00101110}_2 = 46$, and `updateBit(44,1,false)` $= 44\&\texttt{...1101} = 44$ (unchanged). Trace each on paper; the column picture is the whole game.

3.4 Mask construction and bit fields

From the six primitives we assemble the standard recipes for whole *runs* of bits, usually called *masks* or *fields*. A mask of k ones starting at the bottom is `(1 << k) - 1`; shifting it left by l produces ones in positions $l \ldots l + k - 1$. To *extract* the field, shift the number right by l first, then AND with the run: `(n >> l) & ((1 << len) - 1)`. To *clear* the field, AND with the complement of the run placed by l. To *insert* a new value into the field, clear it and OR the shifted value, exactly like `updateBit` but over many bits. These four recipes appear verbatim inside nearly every competitive program that packs several small counters into one integer—the standard example is encoding the k-th move of a chess position or the counts of coins of each type in a game state.

```cpp
// ones in positions l .. l+len-1
int fieldMask(int l, int len)      { return ((1 << len) - 1) << l;    }

int getField(int n, int l, int len){ return (n >> l) & ((1 << len)-1);}

int clearField(int n, int l, int len){ return n & ~fieldMask(l, len); }

int setField(int n, int l, int len, int v){
    return clearField(n, l, len) | (v << l);   // caller guarantees v < 2^len
}
```

Listing 2: Field extraction, clearing, and insertion.

Two debugging habits make field code trustworthy. First, print intermediate masks in binary (a one-line helper with `std::bitset` appears in Section 8) and compare against the intended picture; most field bugs are off-by-one in the length, showing up as one stray high bit. Second, when the field length can reach the full width of the type, remember that `(1 << 32)` is undefined

on 32-bit operands: write `(1LL << len) - 1` in 64-bit arithmetic or special-case `len == 32`. The same care applies to the "all ones within the width" constant `~0u` versus `~0ULL`, a distinction that Section 9 returns to in checklist form.

4 Classic One-Liner Techniques

Every experienced contestant carries a mental table of famous one-liners: expressions whose *effect* is known on sight, even though the binary reasoning behind them takes a moment. This section builds that table from first principles. Each technique gets its identity, an explanation of why it works at the bit level, the canonical C++ form, and notes on where it appears in problems. These one-liners are the atoms of Sections 5–8; there is no faster ROI in competitive programming than knowing them cold.

4.1 Clearing the lowest set bit: `n & (n-1)`

Subtracting one from n borrows from the *lowest* set bit: that bit becomes 0, and every zero below it becomes 1, while all higher bits are untouched. AND-ing with the original therefore erases exactly the lowest set bit and changes nothing else. For example with $n = 44 = \texttt{00101100}_2$ we get $n-1 = \texttt{00101011}_2$, and the AND gives $\texttt{00101000}_2 = 40$: the lowest one (bit 2) is gone. The identity has no branches, no loops, and no data dependence—a perfect contest primitive.

Technique 4.1 (Drop lowest set bit)**.** For $n \neq 0$, the expression `n & (n - 1)` evaluates to n with its lowest set bit cleared. Repeating it k times until the value reaches zero therefore counts the set bits, which is Kernighan's algorithm of Section 4.3, and looping in the other direction enumerates the set bits one by one.

The same identity gives the most elegant power-of-two test. A power of two has exactly one set bit; clearing it must produce zero, and no other positive integer has this property. So n is a power of two exactly when $n > 0$ and `(n & (n-1)) == 0`. The $n > 0$ guard is not decoration: $n = 0$ also satisfies the bit condition, and negative n needs two's complement care, so tests like `isPowerOfTwo` below are written on unsigned types where the statement is literally true.

```
bool isPowerOfTwo(unsigned long long n) {
    return n != 0 && (n & (n - 1)) == 0;
}

// process the set bits of n from lowest to highest
for (unsigned x = n; x != 0; x &= (x - 1)) {
    int pos = __builtin_ctz(x);   // position of current lowest set bit
    // ... use pos ...
}
```

Listing 3: Power-of-two test and set-bit loop via `n & (n-1)`.

A close relative clears the *lowest run of set bits* rather than one bit: `n & (n + 1)` erases the lowest run of consecutive ones (e.g. `00101111` → `00100000`), because adding one turns that run into a single carry bit. Its dual, `n | (n - 1)`, *sets* the whole lowest run of zeros below the lowest one. Both appear as sub-steps of constructive problems ("make the number have one block of ones"), and both are worth adding to your sight-reading table next to `n & (n-1)` itself.

4.2 Isolating the lowest set bit: `n & (-n)`

Where `n & (n-1)` *removes* the lowest set bit, its twin `n & (-n)` *extracts* it as a standalone power of two, often called the *lowbit*. The proof is two lines of two's complement: since $x + (\sim x) = \texttt{1111...1} = -1$, adding one to $\sim x$ yields $-x$; the carry of that addition runs up through the block of ones that sits below the lowest set bit and stops exactly there, so $-n$ agrees with n at

the lowest set bit and has zeros below it. AND-ing n with $-n$ keeps only that bit. Concretely, $n = 44 =$ `00101100` gives $-n =$ `11010100`, and `n & (-n)` $=$ `00000100` $= 4$, which is 2^2, matching the lowest set bit of 44.

Technique 4.2 (Lowbit)**.** For $n \neq 0$, `n & (-n)` returns the lowest set bit of n as a power of two. Its position (the index of that bit) is `__builtin_ctz(n)`, and dividing by it peels the bit off. Combined with `n & (n-1)` you can both read and remove the lowest set bit, which is all that Fenwick-tree updates, subset iteration, and the pairing DP of Section 7 ever need.

Lowbit is the engine of the *Fenwick tree* (BIT): updating the parent chain is `i += i & (-i)` and descending during prefix queries is `i -= i & (-i)`. It also drives the submask enumeration `s = (s - 1) & mask` of Section 6 and the canonical-element trick in subset DPs. One caution before you paste it everywhere: on *signed* types, negation is undefined behavior for `INT_MIN` (Section 2), so write lowbit on `unsigned` or `long long` when values can be arbitrary; and `n & (-n)` of $n = 0$ is 0, which is usually the desired loop-terminating behavior but must be checked when you divide by it.

```
unsigned lowbit(unsigned n) { return n & (-n); }   // e.g. 44 -> 4

// peel bits off n, lowest first:
//   x &= x - 1   removes;   x & (-x)   locates
for (unsigned x = n; x != 0; x -= (x & (-x))) {
    unsigned bit = x & (-x);
    // ... use bit (a power of two) or __builtin_ctz(x) ...
}
```

Listing 4: Lowbit in action: iterate set bits, Fenwick-style walking.

4.3 Counting set bits: popcount in all its forms

Counting the ones in the binary representation of n (*popcount*, Hamming weight, or `POPCNT`) is the most frequently executed bit routine in practice: masks as sets ask "how many elements is this subset?", dp tables ask "how many items did I take?", and constructive problems ask "is the parity even?". There are three canonical ways to compute it, ordered from the most instructive to the fastest, and you should know all three because different situations call for different ones.

The *naive* method shifts and masks one bit at a time, taking $O(\log n)$ steps regardless of the answer: loop k from 0 to the width, add `(n >> k) & 1`. The *Kernighan* method repeatedly applies `n &= (n - 1)`, the lowest-bit-clearing identity, and therefore runs in O(number of set bits)—optimal when inputs are sparse, such as masks with few elements. The *hardware* method calls the compiler builtin `__builtin_popcount(x)` (GCC/Clang for 32-bit `unsigned`) or `__builtin_popcountll(x)` for 64-bit values, which emits a single `POPCNT` machine instruction on any judge CPU from the last decade; C++20 wraps the same thing portably as `std::popcount` from `<bit>`.

```
1  int popcountNaive(unsigned n) {          // O(width)
2      int c = 0;
3      while (n) { c += n & 1; n >>= 1; }
4      return c;
5  }
6
7  int popcountKernighan(unsigned n) {      // O(number of set bits)
8      int c = 0;
9      while (n) { n &= n - 1; ++c; }       // drop lowest set bit
10     return c;
11 }
12 // fastest in practice: __builtin_popcount(x) / __builtin_popcountll(x)
13 // C++20:              std::popcount(x)  from <bit>
```

Listing 5: Three ways to count set bits.

A related counting problem deserves its own spot in your toolbox: *how many set bits are there in total from* 1 *to* n*?* asked in $O(\log n)$ without iterating. Count each bit position independently. Bit k follows a period of 2^{k+1}: it is 0 for 2^k steps, then 1 for 2^k steps. Among the $n+1$ integers $0 \dots n$, there are $\lfloor (n+1)/2^{k+1} \rfloor$ full periods contributing 2^k ones each, plus a partial period contributing $\max(0,\ (n+1) \bmod 2^{k+1} - 2^k)$ ones. Summing this contribution formula over k answers the question for any n up to 2^{60} almost instantly.

```cpp
long long totalSetBits(long long n) {       // sum of popcount(0..n)
    long long total = 0;
    for (int k = 0; k < 60; ++k) {
        long long period = 1LL << (k + 1);  // pattern length of bit k
        long long full   = (n + 1) / period * (1LL << k);
        long long part   = max(0LL, (n + 1) % period - (1LL << k));
        total += full + part;
    }
    return total;
}
```

Listing 6: Total number of set bits written in all integers from 0 to n.

The same *per-bit contribution* viewpoint—stop treating the number as one object, treat each bit independently—is one of the most transferable ideas in this guide. Section 5 reuses it to compute sums of XOR over all pairs, and many "construct the number bit by bit" problems are solved by deciding bit k while holding bits $k+1 \dots$ fixed, feasible precisely because bits do not interact arithmetically except through carries.

4.4 Positions, not counts: ctz, clz, and integer log2

Often you need *where* the bits are, not *how many*. `__builtin_ctz(x)` (count trailing zeros) returns the index of the lowest set bit—it is the position twin of the lowbit. Its mirror `__builtin_clz(x)` (count leading zeros) returns the number of zero bits above the highest set bit, so for a 32-bit `unsigned x` the position of the highest set bit is `31 - __builtin_clz(x)`, which is also $\lfloor \log_2 x \rfloor$. GCC offers the shorthand `__lg(x)` for exactly that quantity. These compile to `BSF`/`LZCNT`/`TZCNT` instructions and are effectively free, which is why binary lifting, segment trees on masks, and "peel bits from the top" greedy scans all use them instead of maintaining a manual loop.

ctz and clz are undefined on zero

`__builtin_ctz(0)` and `__builtin_clz(0)` have no defined result: there is no lowest (or highest) set bit to describe. On x86 they typically return garbage rather than crash, so the bug hides in testing. Guard every call: `x ? __builtin_ctz(x) : 32`, or use `std::countr_zero` (C++20), which is *defined* to return the width for zero. The same applies to the division `x / (x & (-x))`—dividing by the lowbit of zero means dividing by zero.

Table 5 is the reference card. Two entries deserve a remark. `__builtin_parity(x)` answers "is the number of set bits odd?" in one instruction; parity of masks decides winner problems in impartial games and Nim-like DP states where only the parity of a count matters, and using it avoids a popcount-plus-mod that the compiler may or may not fold for you. And `std::bit_width(x)` (C++20) returns the *number of bits needed* to represent x, i.e. $\lfloor \log_2 x \rfloor + 1$ for $x > 0$ and 0 for $x = 0$—the safe replacement for the clz-based log2 that never hits undefined behavior.

Table 5: GCC builtins and their C++20 (<bit>) equivalents. Append ll to any builtin for the 64-bit variant, e.g. __builtin_popcountll. Builtins taking unsigned are UB at 0 where marked.

GCC builtin	C++20	Meaning	At 0
`__builtin_popcount(x)`	`std::popcount`	number of set bits	0
`__builtin_ctz(x)`	`std::countr_zero`	trailing zeros (lowest bit idx)	UB
`__builtin_clz(x)`	`std::countl_zero`	leading zeros	UB
`__builtin_parity(x)`	`std::parity`	popcount mod 2	0
`__lg(x)`	`std::bit_width-1`	$\lfloor \log_2 x \rfloor$	UB/-1
`__builtin_ffs(x)`	—	1+ index of lowest bit	0

4.5 Scaling and rounding by powers of two

Because shifts multiply and divide by two, a family of cheap "modulo and scale" formulas follows, valid for non-negative n and k with `n` an unsigned type. They matter in two directions: writing them is sometimes faster, and more importantly *reading* them—half of all bit-trick code on judges is ordinary arithmetic in disguise, and you must be able to translate both ways instantly.

Table 6: Arithmetic identities for non-negative n, k (`M = (1 << k) - 1` is the low mask). Each row is a two-way translation between arithmetic and bitwise.

Bitwise form	Arithmetic meaning	Typical use
`n & 1`	$n \bmod 2$	parity test
`n & M`	$n \bmod 2^k$	cheap modulo, hashing
`n << k`	$n \cdot 2^k$	doubling chains, DP
`n >> k`	$\lfloor n/2^k \rfloor$	halving, log2
`(n >> k) & 1`	bit k of n	read bit
`(n + M) & ~M`	round n up to multiple of 2^k	alignment, buffers
`n & ~M`	round n down to multiple of 2^k	floor alignment

The rounding rows of Table 6 are the least obvious and worth deriving once. To round n *up* to the next multiple of 2^k, first push it past the boundary with `n + M`, then wipe the low k bits with `& ~M`: if n was already aligned, the addition lands exactly on the next multiple's predecessor and the mask rounds back down to n itself, so the formula is correct in both cases without a branch. For example $n = 37$, $k = 3$ (multiple of 8): $37 + 7 = 44$, and $44\&\ldots11000 = 40 = 5 \times 8$. Both rounding formulas power the address-alignment code in allocators and the bucket arithmetic of open-addressing hash tables, where sizes are powers of two so that `& (size - 1)` replaces `% size`.

Technique 4.3 (Next power of two)**.** To round $n \geq 1$ *up to the nearest power of two*, mark the highest set bit, then flood ones below it and step once more: `n-; n |= n >> 1; ...n |= n >> 32;` then `n++`. Equivalently and more safely, `1ULL << (64 - __builtin_clzll(n - 1))`, or simply `std::bit_ceil(n)` in C++20. Used to size hash tables, segment-tree arrays, and dp dimensions to bit-friendly capacities.

```
unsigned long long nextPow2(unsigned long long n) {   // n >= 1
    n--;
    n |= n >> 1;  n |= n >> 2;  n |= n >> 4;          // flood ones
    n |= n >> 8;  n |= n >> 16; n |= n >> 32;         // down to bit 0
    return n + 1;
}
// 64-bit shortcut:  1ULL << (64 - __builtin_clzll(n - 1))
// C++20:            std::bit_ceil(n)
```

Listing 7: Round up to the next power of two, two ways.

Why the flood works: after `n-`, the highest set bit must become set again in the answer's predecessor, which is nextPow2$(n) - 1$, a number of the form `0111...1`. OR-ing with its own right-shifts propagates the highest bit into every lower position: one shift spreads it over a pair, the pair over a nibble, the nibble over a byte, and so on—$\log_2 64 = 6$ steps for 64 bits. Adding one then produces the clean power of two. The same flood pattern reappears in bit-reversal and in some parity loops, so recognizing "OR with shifted self" as *propagate the top bit downward* is a good reading habit.

4.6 Swapping and reversing bits

Two small tricks complete the single-integer repertoire. To *swap the bits at positions i and j*: check whether they differ; if so, XOR both with a mask that has those two bits set, which flips exactly them. To *reverse all 32 bits*, the classic method walks the bits one by one in $O(32)$; the divide-and-conquer method below does it in five constant-time lines by swapping halves recursively: first all bit-pairs, then nibbles, bytes, 16-bit halves, and finally the two 32-bit halves. Recognize the pattern: each line is "swap fixed-size adjacent groups," implemented with the masks `0x55555555`, `0x33333333`, `0x0F0F0F0F`, and so on.

```
int swapBits(int n, int i, int j) {
    if (((n >> i) & 1) != ((n >> j) & 1))   // differ -> flip both
        n ^= (1 << i) | (1 << j);
    return n;
}

uint32_t reverseBits(uint32_t n) {
    n = ((n & 0x55555555u) << 1)  | ((n >> 1)  & 0x55555555u);
    n = ((n & 0x33333333u) << 2)  | ((n >> 2)  & 0x33333333u);
    n = ((n & 0x0F0F0F0Fu) << 4)  | ((n >> 4)  & 0x0F0F0F0Fu);
    n = ((n & 0x00FF00FFu) << 8)  | ((n >> 8)  & 0x00FF00FFu);
    n = ((n & 0x0000FFFFu) << 16) | ((n >> 16) & 0x0000FFFFu);
    return n;
}
```

Listing 8: Swap two bits, and reverse all 32 bits by divide and conquer.

Where is bit reversal actually needed? FFT implementations index butterfly stages in bit-reversed order; some state encodings want low-endian and high-endian views of the same mask; and reverse can double as a *reflection* operator in problems that ask to mirror a mask's structure. In contests, though, its most common appearance is inside other tricks—for instance, reversing before a ctz turns it into a clz on machines lacking one. The swap-bits helper, meanwhile, shows up in problems that permute positions of a bitmask, and in deserialization of packed fields.

Remark 4.1 (Swap two variables with XOR). The sequence `a ^= b; b ^= a; a ^= b;` swaps `a` and `b` because XOR is its own inverse: after the first line `b` is unchanged, after the second `b` holds `a ^ b ^ b = a`, and the third recovers `b` into `a`. It is a classic interview trick, but in real code it is *slower or equal* to `swap(a, b)` on modern compilers, it reads terribly, and it silently destroys data if you ever apply it to the *same* variable twice (`a ^= a` is zero). Know the trick for reading other people's code; write `std::swap` in contests.

4.7 Gray codes: neighboring states, one bit apart

A *Gray code* orders the integers $0 \dots 2^n - 1$ so that consecutive values differ in exactly one bit. The standard (reflected binary) Gray code is given by the one-liner $g = n \oplus (n >\ > 1)$, and the inverse map is a prefix XOR of the shifted value, computable in $O(\log w)$ doubling steps. Decoding: since g is a XOR of several right-shifted copies of n, collapsing `n ^= n >> 1`, then shift 2, 4, 8, 16 accumulates all copies and recovers n. Consecutive test vectors in hardware and consecutive subsets in branch-and-bound prefer Gray order because stepping between neighbors is a single flip: if a DP or local search recomputes an answer from a mask, Gray enumeration lets it update in $O(1)$ per step instead of recomputing from scratch.

```
unsigned grayEncode(unsigned n) { return n ^ (n >> 1); }

unsigned grayDecode(unsigned g) {          // prefix-XOR collapse
    unsigned n = g;
    n ^= n >> 1;  n ^= n >> 2;  n ^= n >> 4;
    n ^= n >> 8;  n ^= n >> 16;
    return n;
}

// visit all n-bit masks so that each step flips exactly one bit
unsigned g = 0;
for (unsigned step = 0; step < (1u << n); ++step) {
    // ... process mask g ...
    g = (step + 1) ^ ((step + 1) >> 1);
}
```

Listing 9: Reflected binary Gray code: encode, decode, and iterate.

The structural view is illuminating: gray(n) is n with each bit replaced by the XOR of itself and the bit to its *left*—a prefix XOR along the bits. That is why decoding is the inverse prefix XOR, and why the code visits masks in an order where bit k flips whenever bit $k{+}1$ of the counter is set: exactly the tower-of-Hanoi move sequence. This correspondence gives an $O(1)$ answer to "which disk moves on step t of the Hanoi puzzle" (it is `__builtin_ctz(t) + 1`) and explains why Gray enumeration is the canonical way to iterate neighboring masks.

4.8 XOR of a range in O(1)

The XOR of all integers from 0 to n follows a period-four pattern, giving a closed form that then answers *range* queries by the prefix trick: $\text{xor}(l, r) = f(r) \oplus f(l-1)$, since every value below l cancels. The pattern for $f(n) = 0 \oplus 1 \oplus \cdots \oplus n$ depends on $n \bmod 4$: it is n for 0, 1 for 1, $n{+}1$ for 2, and 0 for 3. Derivation intuition: pairs $(4t, 4t{+}1)$ contribute 1, pairs $(4t{+}2, 4t{+}3)$ contribute 1, and the tail restarts the cycle; checking the four residues against small cases is faster than a formal proof and is the standard contest derivation.

```
long long xorUpTo(long long n) {           // 0 ^ 1 ^ ... ^ n
    switch (n & 3) {                       // n mod 4
        case 0: return n;
        case 1: return 1;
        case 2: return n + 1;
        default: return 0;
    }
}
long long xorRange(long long l, long long r) {   // l..r, l >= 1
    return xorUpTo(r) ^ xorUpTo(l - 1);
}
```

Listing 10: in constant time.]XOR of 0..n and of any range [l, r] in constant time.

This function is a building block rather than a destination: it appears inside Nim-variant analysis, in problems that ask for the XOR of all subarray sums' endpoints, and as the "O(1) base case" when a constructive problem reduces to computing XORs of long arithmetic progressions. Whenever you see a constraint like $r \leq 10^{18}$ with XORs of consecutive integers, this is the intended tool; looping would be hopeless, and the pattern is short enough to re-derive on paper in thirty seconds if you forget it.

5 XOR Problem Patterns

XOR problems form the most recognizable family in the bit-manipulation canon. They all exploit one of two faces of XOR: as *cancellation* ($a \oplus a = 0$, everything appearing an even number of times vanishes) or as *reversible bookkeeping* ($a \oplus b \oplus b = a$, so a running XOR is a prefix state that can be undone). This section walks the standard problem ladder in ascending difficulty: the unique-element classics, prefix-XOR range queries, subarray counting, impartial games via Nim and Grundy numbers, and the binary trie that answers "maximum XOR" queries. Master these patterns and most Div. 2 XOR tasks become variations you can classify on sight.

5.1 The single number: everything twice except one

Problem 5.1 (Single number)**.** Given n integers where every value appears exactly twice except one value that appears once, find that value. Constraint-style: n up to $2 \cdot 10^5$, values up to 10^9, and $O(n)$ time with $O(1)$ extra memory.

The naive answers fail the memory or time budget: a hash map uses $O(n)$ memory, and sorting costs $O(n \log n)$. XOR does it in one pass with one integer. Fold all elements together with XOR: each duplicated pair contributes $a \oplus a = 0$ regardless of position, zero is the identity, and the lone element survives untouched. Order does not matter because XOR is commutative and associative—you are literally summing occurrence-parities mod two per bit. This is the cleanest possible showcase of the cancellation face of XOR, and it generalizes immediately: XOR of the whole array answers "what is the element with odd multiplicity" for any multiplicity pattern.

```
int singleNumber(const vector<int>& a) {
    int x = 0;
    for (int v : a) x ^= v;   // pairs cancel; the unique one remains
    return x;
}
```

Listing 11: Single number: one pass, one integer of state.

5.2 Two unique numbers: partition by a set bit

Problem 5.2 (Two single numbers)**.** Every element appears exactly twice except two distinct elements u and v that each appear once. Find both, in $O(n)$ time and $O(1)$ space.

Folding the whole array gives $s = u \oplus v$, which is nonzero precisely because $u \neq v$: it has *at least one set bit*, and every set bit of s marks a position where u and v disagree. Pick any such bit—conventionally the lowest, `low = s & (-s)`—and split the array into two buckets: elements with that bit set, and elements without it. u and v fall into different buckets by construction, while every duplicated pair falls entirely into one bucket (both copies share all bits), so each bucket is an instance of the single-number problem. XOR within buckets recovers u and v. The trick to internalize is not the code but the *move*: use one distinguishing bit of an aggregate XOR to split a search space into two independent subproblems.

```cpp
vector<int> twoUnique(const vector<int>& a) {
    int s = 0;
    for (int v : a) s ^= v;            // s = u ^ v  (s != 0)
    int low = s & (-s);                // lowest bit where u, v differ
    int u = 0;
    for (int v : a)
        if (v & low) u ^= v;           // XOR only the "low bit set" bucket
    return {u, s ^ u};                 // v = s ^ u  by cancellation
}
```

Listing 12: Two unique numbers via a partition bit. See Section 9 for the signed-negation caveat.

5.3 The missing number

Problem 5.3 (Missing number)**.** The integers $0 \dots n$ are given with exactly one missing, in any order. Find the missing value in $O(n)$ time and $O(1)$ space, without numeric overflow.

Two XOR solutions exist and understanding both is useful. The first folds two XORs: $X = 0 \oplus 1 \oplus \cdots \oplus n$ (via the closed form of Listing 10) and Y is the XOR of all given values; then $X \oplus Y$ cancels every present number and leaves the missing one. The second folds *indices with values* in a single loop: XOR the running result with both i and $a[i]$; after the loop, XOR with n as well. Both avoid overflow automatically—XOR never exceeds the bit width, unlike the sum-based variant $\frac{n(n+1)}{2} - \sum a[i]$, which needs care when n is large. The same index-XOR pattern finds a missing or duplicated element in permutations and is the base case of the more general "find two missing" problems, where the partition trick of the previous subsection applies again.

```cpp
int missingNumber(const vector<int>& a) {   // values of 0..n minus one
    int n = (int)a.size();                  // full range is 0..n
    int x = n;                              // fold in n first
    for (int i = 0; i < n; ++i) x ^= i ^ a[i];
    return x;
}
```

Listing 13: Missing number via index-value XOR.

5.4 Every element k times except one

Problem 5.4 (Single number II)**.** Every element appears exactly k times except one element that appears once ($k \geq 2$, often $k = 3$). Find the unique element in $O(n \log C)$ time and $O(1)$ space, where C bounds the values.

XOR cancellation no longer works: $a \oplus a \oplus a = a$ leaves the triples alive. The standard solution changes viewpoint to *per-bit counting*, the contribution technique of Section 4.3. For each bit position, count how many elements have that bit set; every k-fold element contributes k, a multiple of k, while the unique element contributes 0 or 1. So `count mod k` equals the unique element's bit at that position. Assemble the answer bit by bit. The approach handles negative numbers if you run over all 32 bits and treat the result as a signed integer, and it degrades gracefully: the same counting gives you the remainder of each bit modulo k, which is exactly what several "sum of triplets" variants ask for.

```cpp
int singleAmongK(const vector<int>& a, int k) {
    int ans = 0;
    for (int b = 0; b < 31; ++b) {      // assume non-negative values
        int c = 0;
        for (int v : a) c += (v >> b) & 1;
```

```
        if (c % k) ans |= 1 << b;       // survivor's bit b
    }
    return ans;
}
```

Listing 14: Single number among k-tuples, via per-bit counting.

For the $k = 3$ case there is also a famous bit-parallel one-pass formulation with two accumulators `ones`, `twos` tracking which bits have been seen 1 mod 3 and 2 mod 3 times: `ones = (ones ^ v) & ~twos; twos = (twos ^ v) & ~ones;`. It is elegant, but the counting version above is easier to reconstruct under pressure and easier to adapt, which is what contests actually reward. Know that the state-machine version exists, derive the counting version on demand.

5.5 Prefix XOR: range queries and subarray counting

Exactly as prefix sums turn range-sum queries into subtractions, *prefix XOR* turns range-XOR queries into XORs. Define $P_0 = 0$ and $P_i = a_1 \oplus a_2 \oplus \cdots \oplus a_i$. Then the XOR of the subarray $a_l \dots a_r$ equals $P_r \oplus P_{l-1}$, because every element before position l appears twice in the two prefixes and cancels. Building P is one pass; each query is then $O(1)$ with no segment tree at all. The same prefix array supports *update-by-toggling* semantics: XOR-ing a range with a value d is two point-updates in the prefix array of a Fenwick-style structure, since XOR is its own inverse.

```
vector<int> buildPrefixXor(const vector<int>& a) {
    int n = (int)a.size();
    vector<int> P(n + 1, 0);
    for (int i = 0; i < n; ++i) P[i + 1] = P[i] ^ a[i];
    return P;
}
int rangeXor(const vector<int>& P, int l, int r) {   // 0-indexed, inclusive
    return P[r + 1] ^ P[l];
}
```

Listing 15: Prefix XOR: build once, answer range-XOR queries in O(1).

Prefix XOR becomes genuinely powerful when combined with a hash map, mirroring the classic "count subarrays with sum k" pattern: subarray $a_l \dots a_r$ has XOR k exactly when $P_r \oplus P_{l-1} = k$, i.e. $P_{l-1} = P_r \oplus k$. So sweep r from left to right, maintain a map from prefix value to how many times it has occurred among $P_0 \dots P_{r-1}$, and add `freq[P[r] ^ k]` to the answer before inserting P_r. The algorithm is $O(n)$ expected time and handles the count-all-pairs phrasing that judges prefer. The identical template answers "number of pairs of prefixes with XOR in a given set," and its pattern-recognition value is high: *an XOR condition between two positions is a hash-map lookup on prefix XORs.*

```
long long countSubarraysXorK(const vector<int>& a, int k) {
    unordered_map<int, long long> freq{{0, 1}};   // empty prefix
    long long pref = 0, ans = 0;
    for (int v : a) {
        pref ^= v;
        auto it = freq.find(pref ^ k);            // wanted earlier prefix
        if (it != freq.end()) ans += it->second;
        ++freq[pref];
    }
    return ans;
}
```

Listing 16: Count subarrays with XOR equal to k.

5.6 Nim and Grundy numbers: XOR as game theory

Problem 5.5 (Nim)**.** Two players take turns. There are several piles of stones; on each turn a player chooses one pile and removes any positive number of stones from it. The player taking the last stone wins. Given the pile sizes, decide whether the first player wins with optimal play.

The answer is one XOR. Compute the XOR of all pile sizes (the *Nim-sum*); the first player wins iff it is nonzero. The proof is the strategy-stealing heart of combinatorial game theory, and it is short enough to reproduce. If the Nim-sum is zero, every move changes exactly one pile, hence changes the XOR by (old pile) $\oplus$ (new pile) $\neq 0$; so from a zero position every move goes to a nonzero position. If the Nim-sum x is nonzero, let b be its highest set bit; some pile has that bit set, and reducing that pile to `pile ^ x` (which is smaller) makes the Nim-sum zero. So zero positions move only to nonzero, nonzero positions can move to zero, and the terminal position (all piles empty, Nim-sum zero) is a loss for the player to move: nonzero positions are winning, zero positions losing.

```
bool firstWins(const vector<int>& piles) {
    int x = 0;
    for (int p : piles) x ^= p;    // Nim-sum
    return x != 0;
}
```

Listing 17: Nim in one pass: first player wins iff Nim-sum is nonzero.

Nim generalizes through the *Sprague–Grundy theorem*: every finite impartial game under normal play is equivalent to a Nim pile of size $g(\text{state})$, where g (the *Grundy number*) is defined recursively as $g(s) = \text{mex}\{g(s') : s' \text{ a position reachable in one move}\}$ and `mex` is the smallest non-negative integer not in the set. The theorem says a composite game (several independent boards, move in exactly one) is winning iff the XOR of the components' Grundy numbers is nonzero—exactly the Nim condition. In practice you compute Grundy values for one component by memoized DFS over its state graph, then XOR across components. The bits appear because Grundy values are small non-negative integers, `mex` over their successors is a boolean-array scan, and the winning condition is literally a popcount inspection of one XOR: bit manipulation supplies both the state arithmetic and the final decision.

```
int grundy(int state, vector<int>& memo) {   // game-specific moves()
    if (memo[state] != -1) return memo[state];
    bool seen[64] = {};
    for (int s2 : moves(state))              // reachable positions
        seen[grundy(s2, memo)] = true;
    int g = 0;
    while (seen[g]) ++g;                     // mex: first missing value
    return memo[state] = g;
}
// composite game:  first player wins  <=>  XOR of grundy(components) != 0
```

Listing 18: Grundy numbers for one component with memoization, then Nim-style combination.

5.7 The binary trie: maximum XOR queries

Problem 5.6 (Maximum XOR pair)**.** Given n non-negative integers with $a_i < 2^{31}$, find the maximum value of $a_i \oplus a_j$ over all pairs $i \neq j$. Constraints make $O(n^2)$ too slow: think n up to $2 \cdot 10^5$ or 10^6.

The decisive observation is that XOR of two numbers can be *maximized bit by bit from the top*: bit 30 of the answer is 1 iff some pair disagrees at bit 30, and once the top bits of the answer are fixed, lower bits only refine within that choice. Greedy on bits needs a fast structure

that answers "among the inserted numbers, how many have bit b equal to d?"—and the cleanest such structure is the *binary trie* (01-trie): insert every number as a path of 31 binary decisions from the most significant bit down, sharing common prefixes. Nodes at depth d partition the inserted numbers by their top d bits.

To answer `maxXor(x)` for a query value x, walk down from the root trying to keep the answer's bit at 1: at bit b, prefer the child with bit `1 ^ d` where `d = (x >> b) & 1`—the opposite branch—because that makes answer bit b equal to 1; if the opposite child does not exist, follow the same branch and that answer bit stays 0. Building the trie is $O(n \log C)$, each query is $O(\log C)$, and the whole problem is $O(n \log C)$: for $n = 2 \cdot 10^5$ and $C = 2^{31}$ that is about six million node visits, comfortably fast. The same structure answers "maximum XOR subarray" (insert prefix XORs, query each prefix—the maximum of $P_r \oplus P_{l-1}$ is exactly a max-XOR pair between a prefix and all earlier prefixes) and, with per-node counts, "count pairs with XOR below K."

```
struct BinaryTrie {
    struct Node { int nxt[2] = { -1, -1 }; };
    vector<Node> t;
    BinaryTrie() : t(1) {}

    void insert(int x) {
        int v = 0;
        for (int b = 30; b >= 0; --b) {        // top bit first
            int d = (x >> b) & 1;
            if (t[v].nxt[d] == -1) { t[v].nxt[d] = (int)t.size(); t.push_back(Node
    ()); }
            v = t[v].nxt[d];
        }
    }

    int maxXorWith(int x) const {              // best inserted ^ x
        int v = 0, best = 0;
        for (int b = 30; b >= 0; --b) {
            int d = (x >> b) & 1;
            if (t[v].nxt[d ^ 1] != -1) {       // opposite branch exists
                best |= 1 << b;                // answer bit b = 1
                v = t[v].nxt[d ^ 1];
            } else {
                v = t[v].nxt[d];               // forced same branch
            }
        }
        return best;
    }
};
```

Listing 19: Binary trie for maximum-XOR queries over inserted numbers.

Two implementation notes save debugging time. First, fix the bit height to the actual bound of the values (30 for $a_i < 2^{31}$, 59 for 64-bit sums)—a trie too shallow silently drops high bits, one too deep merely wastes a little memory. Second, when the task is a maximum over *pairs*, remember the diagonal: if the query value is itself in the trie, `maxXorWith(x)` may return the self-pair $x \oplus x = 0$; the standard fix is to insert all values, then for each value delete its own path count (or insert all but one, in an offline sweep) before querying. The trie also supports *minimum* XOR (prefer the same branch), k-th XOR queries (store subtree sizes), and persistent versions for XOR on array segments—all built on the same walk-with-opposite-branch greedy.

5.8 Per-bit contribution: sum of XOR over all pairs

Problem 5.7 (Sum of pairwise XOR)**.** Given n integers, compute $\sum_{i<j}(a_i \oplus a_j)$ modulo 10^9+7, for n up to 10^5 and values up to 2^{30}.

Pairwise iteration is $O(n^2)$—too slow. The per-bit contribution technique collapses it to $O(n \log C)$. Process each bit position b independently: within bit b, the XOR of a pair is 1 exactly when the two elements disagree at that bit. If c of the n numbers have bit b set, then $n-c$ do not, the number of disagreeing pairs is $c \cdot (n-c)$, and bit b contributes exactly $c \cdot (n-c) \cdot 2^b$ to the total sum. Sum over bits; done. No pair is ever enumerated, and the formula is the canonical example of *distributing a global quantity over independent bit columns.*

```
long long sumPairXor(const vector<int>& a) {   // values < 2^30
    int n = (int)a.size();
    long long ans = 0;
    for (int b = 0; b < 30; ++b) {
        long long c = 0;
        for (int v : a) c += (v >> b) & 1;
        ans += (c * (n - c)) << b;              // disagreements * weight
    }
    return ans;                                 // take mod if required
}
```

Listing 20: Sum of XOR over all pairs, by per-bit contribution.

This pattern is worth studying beyond its immediate use, because it scales to harder relatives with no new machinery: sum of AND over pairs counts *agreeing-on-one* pairs per bit; sum of OR counts pairs where *at least one* has the bit $(n^2-(n-c)^2$ ordered, adjust for $i \neq j$); sum of XOR over all *subarray* pairs uses the same counting on prefix XORs. The meta-lesson: when an expression is a bitwise function of two large families of objects, never enumerate pairs—enumerate the thirty columns and count, per column, how many object pairs fall into each class. The counts multiply the column's weight, and the sum is the answer.

6 Bitmasks as Sets

The single most important *modeling* idea in bit manipulation is that an integer can represent a set. Fix a small universe of elements numbered $0 \ldots n-1$ (objects, vertices, digits, columns); the presence or absence of element i is stored in bit i of an integer called a *mask*. With $n \leq 20$ a 32-bit `int` suffices, with $n \leq 62$ a `long long`, and with `std::bitset` (Section 8) thousands. Sets stored this way gain superpowers: copy, union, intersect, and membership-test all become single machine instructions, and a mask is one function argument, one dp-table index, one array slot. This section formalizes the representation and then develops the three enumeration patterns that power every mask-based algorithm.

6.1 Set algebra on integers

Table 7 is the vocabulary list; it deserves the same fluency as a foreign-language verb table. Two rows hide subtleties. The *complement* row: `~m` flips *all* width bits, including positions above the universe, so always AND with `FULL` to stay inside the universe—comparing an unmasked complement against a masked set gives wrong disjointness answers. The *subset* row: `(m & u) == m` reads "AND-ing away the elements of `u` does not remove anything from `m`", which is exactly $m \subseteq u$; it is used constantly in dp transitions ("is the required set available?") and in inclusion–exclusion loops of Section 7.

Iterating the *elements* of a set is a two-idiom combination: find the lowest element with the lowbit (`m & (-m)`, its index via `__builtin_ctz`), then remove it (`m &= m - 1`). Repeating until the mask empties visits each element once in increasing index order. This loop replaces `for` over

Table 7: Set operations realized by bitwise expressions on masks (`m` and `u` are masks, i an element index, `FULL = (1 << n) - 1` the universe mask).

Operation	Expression	Notes
membership test	`m & (1 << i)`	nonzero means present
insert element	`m |= (1 << i)`	idempotent
erase element	`m &= ~(1 << i)`	
toggle element	`m ^= (1 << i)`	useful for parity states
union	`m | u`	merge candidate sets
intersection	`m & u`	common elements
difference	`m & ~u`	in m, not in u
complement	`FULL & ~m`	mask off the universe first
subset test	`(m & u) == m`	m is a subset of u
proper subset	`m != u && (m & u) == m`	
disjointness	`(m & u) == 0`	no shared element
size	`__builtin_popcount(m)`	elements of the set
empty test	`m == 0`	
first element	`__builtin_ctz(m)`	index of lowest bit

an `std::set` at a fraction of the cost, and unlike container iteration it survives in tight dp inner loops where cache misses and allocation would be fatal. Always write it as a `for (int m2 = m; m2; m2 &= m2 - 1)` on a *copy* if you need `m` intact afterwards.

```
for (int m2 = m; m2 != 0; m2 &= m2 - 1) {
    int i = __builtin_ctz(m2);   // index of the current lowest element
    // ... process element i ...
}
```

Listing 21: Iterate the elements of a mask, lowest index first.

6.2 Enumerating subsets of a universe

The second pattern enumerates *all* subsets of an n-element universe, which are exactly all 2^n values of an n-bit mask:

```
for (int m = 0; m < (1 << n); ++m) {          // all 2^n subsets
    // optional: work only with subsets of a given size
    // if (__builtin_popcount(m) != k) continue;
    // ... m is a subset: bit i set <=> element i chosen ...
}
```

Listing 22: All subsets of an n-element universe, with size filtering.

This loop is the outer shell of brute force over subsets, of "try every way to split the items" problems, and of the dp tables of Section 7. Two refinements earn their keep. First, complexity discipline: 2^n states with an $O(n)$ or $O(2^n)$ inner sweep is fine for $n \leq 20$–22 and hopeless beyond; whenever n exceeds that, the intended solution is almost never subset enumeration. Second, the *Gosper's hack* refinement: iterate only masks with the same popcount k, one per step, without scanning all 2^n. It computes the next integer with the same number of set bits in $O(1)$ and is the standard tool for "choose exactly k items" loops.

```
int m = (1 << k) - 1;                 // smallest mask with k bits
while (m < (1 << n)) {
    // ... process m (popcount(m) == k) ...
    int c = m & (-m);
    int r = m + c;                    // next block of bits
    m = (((r ^ m) >> 2) / c) | r;     // next mask with same popcount
```

```
}
```

Listing 23: Gosper's hack: visit every mask with exactly k set bits.

Understanding Gosper's formula at the bit level: `c` is the lowest set bit; `m + c` creates a carry that merges the lowest block of ones into a single bit at its top; `(r ^ m)` isolates the changed part, which contains both the moved bit and the vacated positions; shifting right by two and dividing by `c` re-packs the remaining ones to the bottom. You do not need to re-derive it in contest—just recognize the idiom *next-same-popcount* and paste it, testing the boundary `m < (1 << n)` in the loop condition, which also excludes masks that would overflow the universe.

6.3 Enumerating submasks of a mask

The third and most mathematically rich pattern enumerates the subsets *of a given set*: all submasks s of a mask m. The canonical loop runs the submasks in decreasing order using a surprising one-liner:

```
for (int s = m; ; s = (s - 1) & m) {
    // ... process s (guaranteed s & ~m == 0, i.e. s subset of m) ...
    if (s == 0) break;              // stop after handling the empty set
}
```

Listing 24: All submasks s of mask m, in decreasing order, including the empty mask.

Why `(s - 1) & m` yields the *previous* submask in decreasing order: subtracting one borrows from the lowest set bit of s, filling everything below with ones—some of which fall outside m; AND-ing with m then trims those illegitimate bits, and the result is exactly the largest submask of m smaller than s. The loop places the termination test at the *end* on purpose: the empty mask $s = 0$ is a legitimate submask (the empty set) and must be processed before the loop exits, and re-entering from 0 would loop forever. This is one of those rare loops that is easier to get right from the template than to reinvent; memorize the shape.

Technique 6.1 (Total cost of submask iteration)**.** A single mask m with p set bits has 2^p submasks. Summed over *all* 2^n masks of an n-bit universe, the number of (mask, submask) pairs is $\sum_m 2^{\text{popcount}(m)} = \sum_k \binom{n}{k} 2^k = 3^n$: each element is in exactly one of three states—outside the mask, inside the mask but outside the submask, or inside both. Hence "for every mask, for every submask" algorithms run in $O(3^n)$, the signature complexity of set-partition DPs, and the reason such problems constrain $n \le 15$–16 rather than 20.

The 3^n bound is worth internalizing numerically because it decides feasibility on sight: $3^{15} \approx 14$ million is fine, while $3^{20} \approx 3.5$ billion is not. Many problems phrase it as "partition the set into groups" or "pick a representative and recurse on the rest", and their intended solution is exactly the double loop of Listing 24 wrapped in a dp whose dimension is one integer. Whenever you see 3^n in an editorial, this is the loop that produced it.

6.4 Inclusion–exclusion over bitmasks

The final application counts objects by the subsets of a property set they *avoid* or *satisfy*—the classic inclusion–exclusion principle, where the subsets are naturally indexed by masks. The canonical contest example: count the numbers in $[1, x]$ divisible by *at least one* of n given primes. For each non-empty subset S of the primes, the count of numbers divisible by *every* prime in S is $\lfloor x / \prod_{p \in S} p \rfloor$; inclusion–exclusion alternates the sign by the subset size. Iterating subsets as masks gives a clean implementation, with the sign coming from `__builtin_parity` (popcount mod 2) rather than tracking it manually.

```
1 long long countDivisible(long long x, const vector<long long>& p) {
2     int n = (int)p.size();
```

```
    long long total = 0;
    for (int m = 1; m < (1 << n); ++m) {        // every non-empty subset
        long long d = 1;
        for (int m2 = m; m2 != 0; m2 &= m2 - 1) {
            int i = __builtin_ctz(m2);
            d *= p[i];                          // product over the subset
        }
        long long cnt = x / d;                  // divisible by ALL in subset
        total += __builtin_parity(m) ? cnt : -cnt;
    }
    return total;
}
```

Listing 25: divisible by at least one of the primes.]Inclusion–exclusion: count numbers in [1, x] divisible by at least one of the primes.

The same skeleton solves "how many strings avoid all forbidden patterns", "union of rectangles" (dimension by dimension), and "count derangements avoiding positional restrictions"—anywhere a quantity over intersections is easy but over unions is hard. The bitmask contribution is structural: subsets are masks, the product is an inner mask-iteration, and the sign is a parity. When n is large but the number of *relevant* properties is small, this $O(2^n \cdot n)$ loop is the whole solution; when even 2^n is too slow, the properties can often be pre-reduced by removing subsumed divisors (drop p if another prime divides it), a preprocessing step that is itself a pairwise mask-style check.

7 Bitmask Dynamic Programming

Bitmask dynamic programming is the technique that turns "small n" constraints ($n \leq 20$–22) from a warning into a promise. The state of a dp over subsets is an integer whose bits record which elements have been processed; transitions manipulate that integer with the mask algebra of Section 6. This section presents the pattern, the two flagship problems where it is unavoidable (Traveling Salesman and assignment), a canonicalization trick that prevents double counting in pairing DPs, and closes with SOS DP, the subset-sum convolution that upgrades naive submask iteration to $O(n \cdot 2^n)$.

7.1 The pattern

A bitmask dp has three ingredients, and recognizing them in a problem statement is 90% of the work. First, a *universe* of n elements—cities, jobs, items, rows—small enough that 2^n fits in memory and time. Second, a *progress order*: some notion of processing elements one at a time, so that "the set of processed elements" is a meaningful state. Third, a *transition* that, given a processed-set mask and its best value, extends the set by one element at a cost depending only on the mask and the new element. The dp table is then an array indexed by the mask (plus any side-dimension the problem needs), filled in increasing mask order because adding a bit strictly increases the mask as an integer.

The standard skeleton, memorize the shape rather than any single instance:

```
vector<long long> dp(1 << n, INF);   // dp[mask] = best value over
dp[0] = base;                        // orderings/plans of 'mask'
for (int mask = 0; mask < (1 << n); ++mask) {
    // choose the element that gets processed next, or
    // extend mask by one element:
    for (int i = 0; i < n; ++i) {
        if (mask & (1 << i)) continue;          // i already inside
        int nmask = mask | (1 << i);
        dp[nmask] = min(dp[nmask], dp[mask] + cost(mask, i));
```

```
    }
}
// answer = dp[(1 << n) - 1]   (or max over masks, per problem)
```

Listing 26: The universal bitmask DP skeleton.

Complexity bookkeeping: the table has 2^n states; each state tries n extensions, so the loop is $O(n \cdot 2^n)$ time and $O(2^n)$ memory—about $20 \cdot 10^6$ operations for $n = 20$, instant; about 10^9 for $n = 24$, borderline; infeasible at $n = 26$ and beyond unless memory-optimized. That boundary is the practical reading of the famous heuristic *"$n \leq 20$ smells like bitmask dp"*. Memory is often the binding constraint before time: 2^{24} long longs are 128 MB, so problems above $n \approx 23$ demand narrower value types or rolling arrays over popcount layers.

7.2 Traveling Salesman Problem

Problem 7.1 (TSP)**.** Given $n \leq 18$ cities and an $n \times n$ weight matrix w, find the minimum-cost route that starts at city 0, visits every city exactly once, and returns to city 0.

Without the small-n clause, TSP is NP-hard and no polynomial algorithm is known; with it, bitmask dp is the intended solution and runs in $O(n^2 \cdot 2^n)$. Define

$$dp[mask][i] = \text{minimum cost of a path that starts at city 0,}$$
$$\text{visits exactly the cities of } mask, \text{ and ends at city } i,$$

with i required to be in $mask$. The base case is $dp[\{0\}][0] = 0$. The transition appends one new city j outside the mask: $dp[mask \cup \{j\}][j] = \min(\cdot,\ dp[mask][i] + w[i][j])$ for every $i \in mask$. The answer closes the cycle with $\min_i(dp[\text{FULL}][i] + w[i][0])$. The double role of the mask—"where I have been"—and the side dimension—"where I am"—is the blueprint for hundreds of variants: counting routes (replace min with +), longest path over subset DAGs, and feasibility with resource limits (store reachable booleans).

```
const long long INF = 1e18;
int n;                                  // n <= 18
// w[i][j] holds the edge weights
vector<vector<long long>> dp(1 << n, vector<long long>(n, INF));
dp[1][0] = 0;                           // visited = {0}, standing on 0
for (int mask = 1; mask < (1 << n); ++mask)
    for (int i = 0; i < n; ++i) {
        if (!(mask & (1 << i)) || dp[mask][i] == INF) continue;
        for (int j = 0; j < n; ++j) {   // extend by one city
            if (mask & (1 << j)) continue;
            long long nd = dp[mask][i] + w[i][j];
            dp[mask | (1 << j)][j] = min(dp[mask | (1 << j)][j], nd);
        }
    }
long long best = INF;                   // close the tour back at 0
for (int i = 0; i < n; ++i)
    best = min(best, dp[(1 << n) - 1][i] + w[i][0]);
```

Listing 27: TSP in $O(n^2 \cdot 2^n)$: dp over (visited set, current city).

7.3 The assignment problem

Problem 7.2 (Assignment)**.** Assign n people to n jobs, one job per person, minimizing the total cost given $cost[i][j]$. The Hungarian algorithm solves the general case, but with $n \leq 20$ a bitmask dp is simpler to write and to trust.

The key structural insight: when people are assigned in index order, the mask of used jobs *implies* which person is next—person $popcount(mask)$—so a single dimension suffices: $dp[mask]$ is the minimum cost of assigning the first $popcount(mask)$ people to the jobs of $mask$ (as a set, any order of jobs). Transition: person i takes an unused job j, costing $cost[i][j]$. The same "mask encodes a prefix of a sequence" idea handles seating arrangements, schedule packing, and any problem whose natural order fixes half of the state.

```
vector<long long> dp(1 << n, INF);
dp[0] = 0;
for (int mask = 0; mask < (1 << n); ++mask) {
    if (dp[mask] == INF) continue;
    int i = __builtin_popcount(mask);   // next person to place
    for (int j = 0; j < n; ++j) {
        if (mask & (1 << j)) continue;  // job j already taken
        int nm = mask | (1 << j);
        dp[nm] = min(dp[nm], dp[mask] + cost[i][j]);
    }
}
// answer = dp[(1 << n) - 1]
```

Listing 28: over used jobs; the popcount names the next person.]Assignment DP: dp[mask] over used jobs; the popcount names the next person.

7.4 The lowest-bit trick against double counting

A whole family of dp problems counts or optimizes over *partitions into unordered groups*: pair up all elements minimizing total cost, split the set into compatible triples, count perfect matchings. Iterating "choose any two elements" double counts each partition once per ordering of its pairs, and removing duplicates by sorting is messy. The canonical fix is a *canonicalization rule*: process the *lowest-indexed* unpaired element first. Concretely, for dp over masks representing "these elements are already handled", find the lowest set bit $i =$ `__builtin_ctz(mask)` and force every transition to consume i in its first group. Each partition then has exactly one derivation, because any partition's first group must contain the lowest remaining element.

```
vector<long long> dp(1 << n, INF);      // pair up elements of the mask
dp[0] = 0;
for (int mask = 1; mask < (1 << n); ++mask) {
    int i = __builtin_ctz(mask);        // lowest unpaired element:
    dp[mask] = INF;                     // it MUST go in mask's last pair
    for (int j = i + 1; j < n; ++j)
        if (mask & (1 << j)) {
            int rest = mask ^ (1 << i) ^ (1 << j);
            dp[mask] = min(dp[mask], (long long)(a[i] ^ a[j]) + dp[rest]);
        }
}
```

Listing 29: Minimum-cost perfect matching by pairing: each partition is generated exactly once.

Listing 29 solves the classic "split the array into $n/2$ pairs minimizing the sum of xor within pairs" in $O(n^2 \cdot 2^n)$. The same rule handles grouping into triples (two nested loops over companions of the lowest element, $O(3^n \cdot n)$ via the submask bound of Section 6.3) and tiling-like problems where the lowest unfilled cell determines the shape you may place next. When a bitmask dp produces suspiciously wrong counts, the first audit is always: *is each unordered configuration counted once, by a canonical rule such as lowest-bit-first?*

7.5 Broken-profile DP: masks across a frontier

When the objects being processed form a *grid* rather than a set, the state is not "which cells are done" (too many) but "which cells of the frontier interact with the future". Sweep cells in row-major order and keep a mask of the last w cells (a window width w for dominoes, $w+1$ for kings-neighbor problems): this is *broken-profile* (plug) dp. The mask plays the same role—it is the interface between the processed past and the unprocessed future—but transitions are now local shapes that slide the window. A full treatment is beyond this guide's scope; what matters here is the transferable idea that *a mask is a boundary*, and any DP whose past and future interact only through w positions can be solved with $O(\text{rows} \cdot 2^w)$ states. Classic applications: domino tiling of an $n \times m$ grid ($O(n \cdot 4^m)$ with adjacent-transition tables), counting Hamiltonian cycles on narrow grids, and placing pieces with exclusion neighborhoods.

7.6 SOS DP: sums over submasks in near-linear time

Problem 7.3 (Sum over subsets)**.** For every mask m of an n-bit universe, compute $f[m] = \sum_{s \subseteq m} g[s]$, the sum of $g[s]$ taken over every submask s of m. With $n \le 20$, the naive loop over $\sum_m 2^{\text{popcount}(m)} = 3^n$ pairs (about 3.5 billion at $n = 20$) is too slow; the target is $O(n \cdot 2^n)$.

The technique, universally called *SOS DP* (sum over subsets), resembles counting-sort-by-bits. Observe that a submask of m is obtained by turning off an arbitrary subset of m's bits. Build the answer in n passes: after pass i, let $f_i[m]$ be the sum of $g[s]$ over submasks s that agree with m on bits above i (the high bits are already "frozen" to m's). Pass i then allows bit i to be turned off: $f_i[m] = f_{i-1}[m] + f_{i-1}[m \oplus (1 << i)]$ whenever bit i is set in m (otherwise $f_i[m] = f_{i-1}[m]$). The recurrence partitions submasks into "bit i same as m" and "bit i off", exactly a divide-and-conquer over one coordinate, and after all n passes every bit has been allowed to vary, so $f = f_n$ is the full SOS transform. In-place implementation with a single array (loop bits outer, masks inner, ascending) makes the space $O(2^n)$.

```
// g[mask] holds the value of the exact set 'mask'
vector<long long> f = g;                    // will become SOS(g)
for (int i = 0; i < n; ++i)                 // allow bit i to vary
    for (int m = 0; m < (1 << n); ++m)
        if (m & (1 << i))
            f[m] += f[m ^ (1 << i)];
// now f[m] = sum of g[s] over every submask s of m
```

Listing 30: SOS DP: the value at each mask becomes the sum of g over all its submasks, in $O(n \cdot 2^n)$.

Problem 7.4 (Pairs with AND zero)**.** Given n integers below 2^k, count ordered pairs (i, j) with $a_i \,\&\, a_j = 0$. With $k \le 20$ and $n \le 10^6$, checking pairs is hopeless; SOS makes it linear in the value universe.

The application shows why SOS is worth knowing. Build $cnt[v]$ = number of array elements equal to v, and run SOS to get $sub[m]$ = number of elements that are submasks of m. An element a_j ANDs to zero with a_i exactly when a_j is a submask of $\sim a_i$ (within k bits), so the answer is $\sum_i sub[\text{FULL} \oplus a_i]$ —since FULL $\oplus\, a_i$ is the complement of a_i inside the universe—minus the self-pairs counted when $a_i = 0$. The mirror problem (pairs with OR covering the universe, superset sums) is the same dp with the roles of 0 and 1 swapped—iterate `m ^ (1 << i)` on the *unset* bits, or simply complement the universe at the start. Between them, SOS DP plus its superset twin is the standard tool for any problem of the form "for each mask, aggregate over related masks", where "related" means differing only in a subset of positions.

8 std::bitset

Definition 8.1 (std::bitset)**.** `std::bitset<N>` is a fixed-size sequence of N bits with word-parallel implementations of the bitwise operators, provided by `<bitset>`. The size N is a template parameter chosen at compile time; operations on the whole bitset run in $O(N/64)$ machine words, so a 2000-bit bitset operation costs about 32 word instructions—roughly sixty times cheaper than a hand-written per-bit loop.

That word-parallelism is the entire reason `bitset` earns a place in contest code: it divides any per-bit-loop complexity by the machine word size, conventionally written as the "/64" in complexities like $O(V^3/64)$ for transitive closure. Wherever an algorithm's inner loop is "apply a bitwise operation to all positions"—reachability propagation, sieve marking, dp over column masks—replacing the loop with one `bitset` expression is a constant-factor win large enough to change feasibility classes at contest scale.

8.1 The API

Table 8: Essential `std::bitset<N>` members. GCC additionally provides `_Find_first` and `_Find_next`, which the standard does not guarantee but every major judge compiles.

Member / operator	Meaning
`b[i]`, `b.test(i)`	read bit i (no bounds check / with check)
`b.set(i)`, `b.reset(i)`	set / clear bit i (no arg: all bits)
`b.flip(i)`	toggle bit i (no arg: all bits)
`b.count()`	popcount of the whole bitset
`b.any()`, `b.none()`	is at least one / no bit set
`b & c`, `b | c`, `b ^ c`, `~b`	whole-object bitwise ops
`b <<= k`, `b >>= k`	shift the whole bitset
`b == c`, `b != c`	comparison
`to_ullong()`, `to_string()`	export (mind overflow for $N > 64$)
`_Find_first()`	index of lowest set bit (N if none)
`_Find_next(i)`	next set bit after i (N if none)

Table 8 lists the surface you need. Three practical notes keep bitsets pleasant. First, they are *fixed-size at compile time*: pick N generously from the constraints (a power of two above the maximum index) because resizing means changing the type. Second, printing for debug is trivial—`cout << b` emits the full binary string, the instant-visualization helper promised in Section 3.3. Third, for thousands of bitsets, prefer `vector<bitset<N>>` over `vector<vector<bool>>`: the latter is a notorious bit-packed slowpoke, while the former is a flat, cache-friendly array of machine words.

```
bitset<2000> b;                        // 2000 bits, all zero
b.set(3); b.set(41); b.set(100);
for (int i = b._Find_first(); i < (int)b.size();
     i = b._Find_next(i)) {
    // visit i = 3, then 41, then 100
}
```

Listing 31: Iterate set bits of a bitset with the GCC extensions.

8.2 Application: reachability and transitive closure

Problem 8.1 (Transitive closure)**.** Given a directed graph with $V \le 2000$ vertices, compute for every vertex the set of vertices reachable from it. Explicit per-edge iteration costs $O(V \cdot (V+E))$—up to $4 \cdot 10^9$ operations at $V = 2000$, too slow.

The *bitset Floyd–Warshall* reaches $O(V^3/64) \approx 1.25 \times 10^8$ word operations, well within limits. Maintain `reach[v]` as a bitset whose bit u says "u is reachable from v"; initialize from the adjacency lists, setting bit v in `reach[v]` itself if paths of length zero count. The Floyd–Warshall relaxation becomes a single OR: if v reaches k and k reaches the set `reach[k]`, then v reaches all of `reach[k]`, so `reach[v] |= reach[k]`. Order the k-loop outermost exactly as in ordinary Floyd–Warshall over booleans.

```
const int V = 2000;
bitset<V> reach[V];
// initialize: for every edge v -> u:  reach[v].set(u);
//             (optionally reach[v].set(v) for length-zero paths)
for (int k = 0; k < V; ++k)
    for (int v = 0; v < V; ++v)
        if (reach[v][k])              // v -> k exists so far
            reach[v] |= reach[k];     // inherit all of k's reachability
// reach[v] is now the full reachable set of v
```

Listing 32: Transitive closure in $O(V^3/64)$ with bitsets.

The same reachability bitset answers queries in $O(1)$ per pair (`reach[v][u]`), counts successors with `count()` (the "maximum number of reachable vertices" and dominant-vertex problems), and composes with the mask algebra of Section 6: two-hop paths, mutual reachability (`reach[v] & reachInv[v]`), and SCC detection via intersecting forward and backward sets all become single expressions. When V exceeds a compile-time bound (say 10^4, with $V^3/64 \approx 1.6 \times 10^{11}$ too slow), the graph truth is that bitsets rescue medium sizes only—beyond that the problem demands SCC condensation or bitset-friendly algorithms on the condensation, which is exactly the "did the constant factor buy you a class change?" audit to run before committing.

8.3 Application: sieve and dp acceleration

Two further standard uses round out the tool. A prime sieve over $N = 10^8$ fits in a `bitset<N>` of about 12.5 MB (versus 100 MB for a `vector<char>`), and even though the inner composite-marking loop below writes one bit at a time, the memory saving alone lifts cache behavior noticeably; the fully word-parallel variant keeps a raw array of `uint64_t` blocks and ORs precomputed patterns per prime. The showcase for word-parallelism is dp reachability: the row of a "which sums are achievable" boolean table is a bitset, and adding item w is `sum |= sum << w`, shifting the whole table in one statement. The subset-sum-of-10^5-items-at-10^5-sums problem drops from 10^{10} boolean writes to 10^5 word-shifted ORs over $10^5/64$ words each—about 1.5×10^8 word ops, fast enough, and one line of code.

```
const int N = 100000000;
bitset<N> composite;
composite[0] = composite[1] = true;
for (int p = 2; p * p < N; ++p)
    if (!composite[p])
        for (long long q = 1LL * p * p; q < N; q += p)
            composite[q] = true;

bitset<100001> sum;                 // which sums are achievable
sum[0] = true;
for (int w : weights)               // 0/1 knapsack reachability
    sum |= sum << w;                // one line per item
```

Listing 33: Sieve of Eratosthenes and subset-sum reachability with bitsets.

Both snippets reward a second look at their complexity bookkeeping. The sieve loop is $O(N \log \log N)$ *bit writes*; packing them into machine words is what the raw-`uint64_t` variant buys, while the bitset version's win is memory and cache. The knapsack line is the opposite:

whole-object `sum « w` is natively word-parallel, each statement shifting 100001/64 words at once. It inherits a subtlety worth naming: bits that slide past the end of a `bitset` are discarded, which here is exactly the capacity bound—correctness and overflow handling agree, one of the rare places where silent truncation is precisely the intended semantics.

9 Pitfall Compendium

Every technique in this guide has a shadow side where the elegant expression becomes undefined behavior or plain wrong arithmetic. This section collects the traps in one place so that a pre-contest re-read takes five minutes. The organizing principle: C++ bitwise code fails in exactly three ways—operations on values that have no legal answer, operations on widths you did not mean, and expressions that group differently than they read. Each pitfall below names its class.

Shifting by too much, or by a negative count

`x << 32` with 32-bit `x` is undefined behavior—not "gives zero". The count must be strictly less than the width of the *left operand*, and the left operand's type decides the width: `1 << 35` is UB because `1` is `int`. Write `1ULL << 35` or `(unsigned long long)1 << 35` when the count can reach or exceed 31. Negative shift counts are UB as well, and they appear more often than you would guess through variables like `31 - k` when `k` over-runs. The same rule applies on the right: `x >> w` is as illegal as `x << w`.

Signed right shifts and negation of the minimum

Pre-C++20, `x >> 1` on a negative signed `x` is implementation-defined (arithmetic shift in practice), and division `x / 2` rounds toward zero instead, so *they disagree on odd negatives*: `-3 >> 1` $= -2$ but $-3/2 = -1$. Since C++20 the shift is defined as arithmetic, but the disagreement with division remains. Separately, `-INT_MIN` is UB (no positive partner exists), which poisons `x & (-x)` and any `abs`-based code when `INT_MIN` is reachable; move to `unsigned` or `long long` before negating.

Precedence: bitwise binds looser than comparison

`x & 1 == 0` parses as `x & (1 == 0)` $= 0$; the intended `(x & 1) == 0` needs its parentheses. The general order to respect: arithmetic, then shifts, then comparisons, then `&`, then `^` (yes, *below* AND), then `|`, then `&&`, then `||`. A source-level habit that eliminates the entire class: parenthesize every bitwise expression that also contains a comparison or mixes two bitwise operators. Compilers warn (`-Wparentheses`) for the `&`/`|` against `==` case—read those warnings.

Zero inputs to ctz, clz, and lowbit division

`__builtin_ctz(0)` and `__builtin_clz(0)` are undefined; `x / (x & (-x))` divides by zero at `x = 0`; and `_Find_first()` on an empty bitset returns `size()`, which is safe only if your loop checks `i < N`. Every one of these lurks behind an input like "all elements equal" or "empty set". Either guard explicitly or switch to the C++20 functions `std::countr_zero`/`std::countl_zero`, which return the width for zero by definition.

Masks that outgrow int

A 40-item universe does not fit in `int`: `1 << 40` shifts a 32-bit constant and is UB, and even `(1LL << 40) - 1` must then be *stored* in `long long`, not truncated back. Audit three places when moving to 64-bit masks: loop bounds (`m < (1LL << n)`), helper signatures (masks as `long long` throughout, not `int`), and any table indexed by mask (`vector<long`

`long> dp(1LL << n)`). Mixing `int` masks with `long long` masks in one expression silently drops the high half.

~ and unsigned promotion

`~x` on an 8-bit `char` promotes to `int` first and flips 32 bits, so comparing `~x == 5` or storing the result into a narrow type behaves unexpectedly; build inverted masks in the width you need (`~0u`, `~0ULL`) and AND with `FULL` before comparing. In the same family: `1 << i` inside a `long long` expression is still a 32-bit shift if `i` can reach 31—the `1LL` must be at the *shift*, not later in the expression.

Counting and comparing signed values as masks

`__builtin_popcount(-1)` returns 32 on a 32-bit signed input—the popcount of the two's complement bit pattern, not an error—so popcounting negative values usually signals a logic bug upstream. And the sign-comparison promotion of Section 2 applies inside loops: `for (unsigned i = mask; i >= 0; -i)` never terminates, because unsigned values cannot go below zero. Keep loop counters signed; keep masks unsigned; keep them in separate variables.

A final meta-pitfall deserves emphasis because it is the one that survives code review: *testing only on pretty examples*. Bit code is extremely sensitive to boundary positions—bit 0 versus bit 1, inclusive versus exclusive field ends, n versus $n-1$ bits in a mask. The reliable test suite for any bit function has four members: 0 itself, 1 (single lowest bit), the all-ones value of the width, and one random value with two adjacent set bits adjacent to the boundaries. Functions that pass those four almost never fail in contest; functions that were only tried on `0b101100` regularly do.

10 Cheat Sheet and Practice Roadmap

10.1 The one-page cheat sheet

Table 9 compresses the entire guide into the expressions you will actually type. Read the "Meaning" column as the sight-reading translation you should be able to perform in both directions: given the code, say the meaning; given the meaning, write the code without hesitation.

Table 9: Bit-manipulation quick reference (all values non-negative unless noted).

Expression	Meaning	Notes / source
`n & 1`	parity of n	even/odd test
`n & (n - 1)`	drop lowest set bit	power-of-two test with $n > 0$
`n & (-n)`	isolate lowest set bit	lowbit; Fenwick trees
`n >> i & 1`	bit i of n	read one bit
`n | (1 << i)`	set bit i	
`n & ~(1 << i)`	clear bit i	
`n ^ (1 << i)`	toggle bit i	
`(1 << k) - 1`	k lowest bits all one	field masks
`(n >> l) & ((1 << len) - 1)`	field $[l, l{+}len)$	extract bits
`n & ((1 << k) - 1)`	$n \bmod 2^k$	cheap modulo

Expression	Meaning	Notes / source
`(n + M) & ~M`	round up to 2^k	$M = 2^k - 1$
`n & (n + 1)`	clear lowest run of ones	
`n | (n - 1)`	set bits below lowest one	
`n << k`, `n >> k`	$n \cdot 2^k$, $n/2^k$	unsigned n only
`a ^ b ^ b`	a	XOR is self-inverse
`__builtin_popcount(x)`	number of set bits	`ll` for 64-bit
`__builtin_ctz(x)`	index of lowest set bit	UB at $x = 0$
`__builtin_clz(x)`	leading zeros	$\lfloor \log_2 x \rfloor = 31-$ it
`__builtin_parity(x)`	popcount mod 2	incl.–excl. signs
`gray(n) = n ^ (n >> 1)`	Gray code	neighbors differ in 1 bit
`xorUpTo(n)`	$0 \oplus 1 \oplus \cdots \oplus n$	period 4; range via prefix
`s = (s - 1) & m`	previous submask of m	total 3^n over all m
`(m & u) == m`	$m \subseteq u$	subset test
`FULL & ~m`	complement of m	mask to the universe
`m != 0 && !(m & (m-1))`	m is a power of two	set-size 1
`dp[m | (1 << i)]`	extend subset dp	$O(n \cdot 2^n)$
`__builtin_ctz(m)` in dp	canonical lowest element	kills double counting
`f[m] += f[m ^ (1 << i)]`	SOS DP	submask sums
`reach[v] |= reach[k]`	bitset closure	$O(V^3/64)$
`sum |= sum << w`	knapsack reachability	bitset dp

10.2 Practice roadmap

Techniques stick when each one is exercised against real judge problems within a day or two of reading. The list below maps this guide's sections to well-known problems; all are reachable from standard judges (CSES, Codeforces, AtCoder, LeetCode) and are ordered so that each row assumes only the rows above it.

- **Foundations and one-liners** (Sections 2–4): CSES *Bit Strings*, *Exponentiation* (for shift-adjacent doubling intuition); LeetCode *Number of 1 Bits*, *Power of Two*, *Reverse Bits*, *Single Number*; CSES *Counting Bits*. Goal: write every Table 9 row from memory.
- **XOR patterns** (Section 5): LeetCode *Single Number II*, *Missing Number*, *Subarray XOR Equals K* (prefix-XOR map); Codeforces educational rounds' "MEX vs XOR" style tasks for parity arguments; AtCoder ABC tasks tagged XOR for constructive practice; CSES *Gray Code*. Goal: classify an unseen XOR task within one minute into cancellation, prefix, or bit-greedy.
- **Masks and enumeration** (Sections 6–6.3): CSES *Apple Division* and *Splitting a Lab* (subset brute force); any "count numbers divisible by at least one of the given k numbers" task (inclusion–exclusion over masks). Goal: submask loop and Gosper's hack written without reference.
- **Bitmask DP** (Section 7): CSES *Elevator Rides* and *Hamiltonian Flights*; AtCoder DP contest task *O – Matching* (assignment dp); classic TSP from any library judge; Codeforces tasks on pairing up elements to minimize total XOR, for the lowest-bit canonicalization. Goal: skeleton of Listing 26 adapted to a new problem in ten minutes.
- **SOS and bitsets** (Sections 7 and 8): Codeforces problems tagged "sum over subsets" (e.g. the *Special Pairs* family); reachability-closure tasks with $V \le 2000$; CSES *Grid Paths*-adjacent bitmask-row dp. Goal: recognize when a per-bit loop is actually a /64 word-parallel win.

- **Tries and bit-greedy** (Section 5): LeetCode *Maximum XOR of Two Numbers*; Codeforces maximum-XOR-subarray variants. Goal: full trie from Listing 19 reproduced from scratch.

10.3 Closing advice

Three habits separate contestants who "know" these techniques from those who apply them reliably. First, translate constraints into bit-budgets automatically: $n \leq 20$ means 2^n masks are on the table, values below 2^{30} mean 30-bit tries, and a modulus near 2^{63} hints at unsigned tricks. Second, when stuck on a bitwise constructive problem, decide the answer bit by bit from the top, maintaining only what past decisions constrain—this greedy skeleton solves an enormous family of tasks whose editorials look unrelated. Third, keep the pitfalls of Section 9 on a checklist you read before submitting anything containing shifts: the bugs there do not announce themselves in testing, and five minutes of review is the cheapest insurance in competitive programming. The techniques in this guide are old, stable, and universal—which is exactly why they are worth knowing perfectly rather than approximately.

Where to go next. For reference material beyond this guide, the *CP-Algorithms* website (cp-algorithms.com) has rigorous articles on every topic touched here, from SOS DP to bitmask tricks over subsets, each with proofs and tested code. The KACTL (KTH Algorithm Competition Template Library) contains contest-hardened implementations of the same primitives and is a masterclass in writing short, correct bit code. For the hardware level beneath the builtins—why popcount is one instruction and how compilers fold shifts—Henry Warren's book *Hacker's Delight* remains the definitive tour of bit-level computation. Reading one chapter of any of these per week, alongside the practice roadmap above, will move every trick in this guide from "familiar" to "reflex" within a season.